

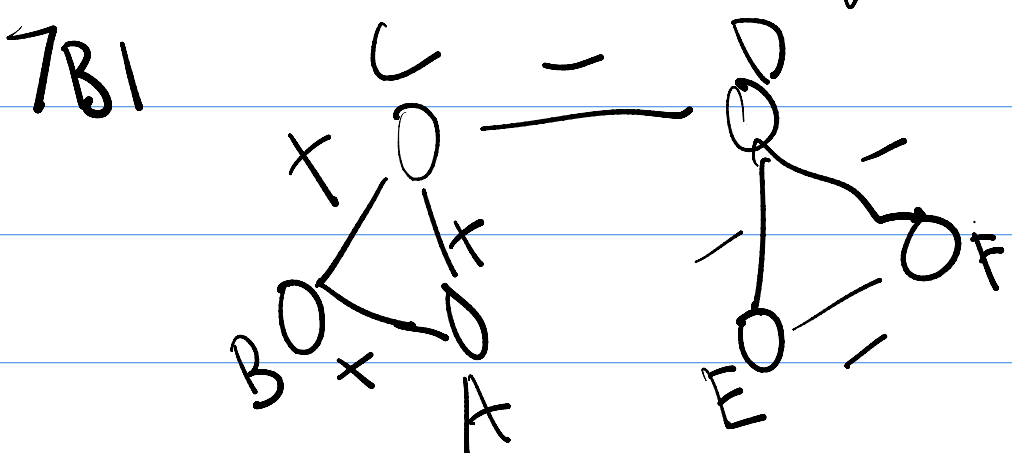
Network - ex 07

- 2.A.1
- ① $A \pm B, B \pm C, A \pm C$ (all positive)
 - ② $A \pm B, B \pm C, A = C$ (two positive, one negative)
 - ③ $A = B, B = C, A \pm C$ (one positive, two negative)
 - ④ $A = B, B = C, A = C$ (all negative)

① and ② is stable, the first one is all coalition, third one a coalition happens because there are friends that share common enemy

③ unstable, a person has two friends that hate each other

④ unstable, no coalition could be made



① A, B, C is balanced under strong and weak balance

② D, E, F is unbalanced under strong balance (product is -1) Weak balance only forbids (+, +, -) triangles, so all negative is weakly balanced

Strong balance

Forbids: $(+, +, -)$
 $(-, -, -)$

③ Strong balance, must flip one of the edges at $\{D, E, F\}$ to positive

Weak balance, allows more than two camps $\{A, B, C\}, \{D, E, F\}$ three mutual enemies forms three separate factions

④ Strong balance predicts an alliance forms (at most two camps) Weak balance, multi-polar rivalry is stable.

Weak balance

Forbids: $(+, +, -)$

7.B.2

```
study > snippets > network-ex07 > 7.B.2.py > ...
1 import networkx as nx
2 from itertools import combinations
3
4 G = nx.Graph()
5 G.add_edges_from([(1,2),(1,3),(2,3),(4,5),(4,6),(5,6),(1,4)], sign=1)
6 G.add_edges_from([(2,5),(3,6),(1,5),(2,6),(3,4)], sign=-1)
7
8 def triangle_sign(G, t):
9     a, b, c = t
10    return G[a][b]["sign"] * G[b][c]["sign"] * G[a][c]["sign"]
11
12 triangles = [t for t in combinations(G.nodes, 3)
13              if all(G.has_edge(u, v) for u, v in combinations(t, 2))]
14 balanced = sum(1 for t in triangles if triangle_sign(G, t) > 0)
15
16 #which triangles are balanced and which are not
17 balanced_triangles = [t for t in triangles if triangle_sign(G, t) > 0]
18 unbalanced_triangles = [t for t in triangles if triangle_sign(G, t) < 0]
19
20 print(f"Balanced triangles: {balanced_triangles}")
21 print(f"Unbalanced triangles: {unbalanced_triangles}")
22
23 #which are the two camps partitions based on weak balance
24 partition = nx.algorithms.community.kernighan_lin_bisection(G, max_iter=100)
25 print(f"Camp 1: {partition[0]}")
26 print(f"Camp 2: {partition[1]}")
```

```
> python3 7.B.2.py
Balanced triangles: [(1, 2, 3), (1, 2, 5), (1, 3, 4), (1, 4, 5), (2, 3, 6), (2, 5, 6), (3, 4, 6), (4, 5, 6)]
Unbalanced triangles: []
```

```
Camp 1: {1, 3, 4}
Camp 2: {2, 5, 6}
python3 7.B.2.py
Balanced triangles: [(1, 2, 3), (1, 2, 5), (2, 3, 6), (2, 5, 6), (3, 4, 6), (4, 5, 6)]
Unbalanced triangles: [(1, 3, 4), (1, 4, 5)]
Camp 1: {3, 4, 6}
Camp 2: {1, 2, 5}
```

7C1 WWI Alliance Network

- ① Balanced as allies forms $(+, +, +)$ and central power forms $(+, +, +)$
- ② Yes as Camp 1 (allies) are friends with each other and is also hates the camp 2 (central power) and vice versa.
- ③ A new country with positive ties to both camps creates $+, +, -$. It will create the triangle unstable

4. Italy initially had a positive tie to Germany while Allies-Germany was negative, straddling two hostile camps. The $(+, +, -)$ tension is exactly what balance theory predicts must resolve, and the historical switch in 1915 eliminated the unbalanced triangles.

7C2